Architecture logicielle et objets répartis

Séance 5 Atelier pratique : modéliser des couches, puis retourner le miroir vers son projet
Nelly · Université JFN · projet `archi-repartis-nelly`

Partie A Cas fourni : plateforme de réservation de salles

Le diagramme en couches figure page suivante. Le tableau ci-dessous détaille le rôle et les limites de chaque couche.

A.1 Les couches, leurs responsabilités et leurs limites

Couche	Rôle en une phrase	Responsabilités concrètes	Ce qu'elle ne doit PAS faire
Présentation (vues web, `ReservationController`, `AdminController`)	Traduire les interactions des enseignants et de l'administrateur (HTTP, formulaires) en appels métier, puis traduire les résultats et les refus métier en pages ou en réponses.	• Afficher les disponibilités et le formulaire de demande (salle, créneau, effectif). • Vérifier le **format** des saisies : champs présents, date valide, effectif entier positif. • Convertir un refus métier (« créneau occupé », « capacité insuffisante ») en message lisible ou en code HTTP (409, 422).	• Décider si deux créneaux se chevauchent ou si la capacité suffit : ce sont des règles de gestion. • Écrire du SQL ou ouvrir une connexion à la base. • Envoyer le courriel de confirmation (voir A.3). • Être le seul endroit où l'on vérifie qu'un utilisateur est administrateur : masquer un bouton n'est pas une autorisation.
Métier / application (`ReservationService`, `AdministrationService`, port `NotificationPort`)	Appliquer les règles de réservation, quel que soit le canal qui les déclenche : web, tâche planifiée ou future application mobile.	• Refuser une demande dont le créneau chevauche une réservation existante sur la même salle (intervalles semi-ouverts : 10h–12h ne chevauche pas 12h–14h). • Refuser une demande dont l'effectif dépasse la capacité de la salle. • Enregistrer la réservation de façon atomique, sans qu'une deuxième demande concurrente puisse passer entre la vérification et l'écriture, puis déclencher la confirmation via `NotificationPort`. • N'autoriser l'annulation qu'au rôle administrateur.	• Connaître HTTP : pas de `HttpServletRequest`, de session, de code de statut, de HTML ni de JSON. • Connaître le nom des tables ou des colonnes de la base partagée. • Dépendre d'une bibliothèque SMTP précise : elle passe par une interface (port).
Accès aux données (`SalleRepository`, `ReservationRepository`) + infrastructure (`AdaptateurSMTP`)	Stocker et restituer les salles et les réservations, en isolant le reste du système du schéma physique de la base partagée.	• Fournir des requêtes ciblées : `chercherChevauchements(salle, début, fin)`, `trouverSalle(id)`. • Convertir les lignes des tables partagées en objets `Salle` et `Reservation` : c'est le **seul** endroit qui connaît les noms de colonnes de l'équipe Emplois du temps. • Gérer les transactions et le verrou (ou la contrainte d'exclusion) qui empêchent une double réservation.	• Décider si une réservation est acceptable : pas de règle de capacité cachée dans une procédure stockée ou un trigger. • Envoyer des courriels ou connaître l'utilisateur web connecté. • **Modifier** les tables de salles, qui appartiennent au système Emplois du temps : l'accès à ces tables est en lecture seule.

A.2 Le piège du cahier des charges

Le style introduit silencieusement. La phrase « les données de salles et de réservations sont stockées dans une base relationnelle partagée avec le système de gestion des emplois du temps » ajoute aux couches une **architecture centrée sur les données, de type dépôt partagé** (*shared repository*). Le magasin est passif : chaque application décide seule quand elle lit et quand elle écrit. Ce n'est donc pas un tableau noir. Hohpe et Woolf (*Enterprise Integration Patterns*, 2003) décrivent ce cas comme un anti-patron d'intégration : l'**integration database**. Les deux applications s'échangent de l'information par les tables plutôt que par une API, sans que personne l'ait décidé explicitement.

Le risque concret. La plateforme de réservation dépend alors du **schéma physique** d'une base qu'elle ne possède pas. Supposons que l'équipe Emplois du temps fasse évoluer sa table de salles sans prévenir.

- **Panne visible.** Elle renomme `salles.capacite` en `capacite_max`, ou remplace la clé `id_salle INT` par un code `"B201"`. Dès la mise en production, la requête de `SalleRepository` échoue (« column not found ») : plus aucun enseignant ne peut réserver, et l'incident vient d'un changement dont l'équipe Réservation n'a jamais entendu parler.
- **Corruption silencieuse, plus grave.** La colonne garde son nom mais change de sens. Par exemple, `capacite` devient « places assises en configuration examen », soit environ la moitié. Ou bien une colonne `active = false` sert désormais à fermer les salles en travaux. Ou encore les horaires passent en UTC alors que la plateforme raisonne en heure de Douala (UTC+1). Aucune erreur n'apparaît, mais la plateforme accepte des réservations dans des salles fermées ou trop petites, et le décalage d'une heure fausse le contrôle de chevauchement : deux cours peuvent être placés dans la même salle au même moment.

Le cahier des charges laisse aussi une question ouverte, qui relève du même couplage : un cours planifié par le système Emplois du temps compte-t-il comme une « réservation existante » à contrôler ? Si oui, la règle métier dépend des données d'une autre équipe.

Parade proposée. Ne lire les salles qu'au travers de **vues SQL contractuelles** (ou d'une API) possédées et versionnées par l'équipe Emplois du temps. L'application utilise un compte en lecture seule sur ces données, et un test de contrat vérifie le schéma attendu à chaque déploiement. `SalleRepository` joue le rôle de couche anti-corruption : un changement de schéma ne touche qu'une classe, et il est détecté avant la production.

A.3 Le courriel envoyé depuis le contrôleur

Pourquoi c'est une violation de couche. « Toute réservation confirmée donne lieu à un courriel de confirmation » est une **règle de gestion** : elle découle de l'événement métier *réservation enregistrée*, quel que soit le canal qui l'a produit. Si le contrôleur envoie le courriel lui-même, il fait deux choses interdites :

1. Il porte une règle métier, qui ne vit plus dans la couche métier.
1. Il saute la couche métier pour appeler directement l'infrastructure SMTP : la présentation dépend d'une bibliothèque de messagerie.

S'y ajoute un défaut de cohérence. Le contrôleur ne sait pas si la transaction a réellement été validée. Il peut envoyer une confirmation pour une réservation finalement annulée par un conflit concurrent, ou ne rien envoyer si une exception survient après l'écriture.

Ce que cela rend difficile pour le rappel nocturne. La tâche planifiée qui envoie les rappels le matin même ne passe jamais par le contrôleur web. Pour envoyer un courriel, l'équipe n'a alors que de mauvaises options :

- **Dupliquer** la composition et l'envoi du message (gabarit, configuration SMTP, gestion des échecs) dans la tâche. Il y a dès lors deux chemins de code qui divergeront : un changement de gabarit ou de serveur sera fait dans l'un et oublié dans l'autre.
- **Contourner** en appelant sa propre API HTTP depuis la tâche, ou en instanciant un contrôleur avec une fausse requête. La tâche devient dépendante du serveur web.

- **Extraire après coup** l'envoi vers la couche métier. C'est la bonne solution, mais elle arrive tard, avec le risque de casser le flux existant.

Le raccourci empêche aussi de tester la règle « une réservation confirmée déclenche un courriel » sans démarrer la pile web.

Partie B ARCHITECTURE.md : état réel du projet fil rouge

Ce document décrit l'architecture telle qu'elle est dans le code au commit 1c23aca, et non telle qu'elle aurait dû être. Il porte sur les modules /socket, /rmi et /rest. Le schéma annoté figure page précédente : chaque violation citée ici (repères S, R, E) y est annotée directement.

1. Choix de découpage

Le schéma utilise **quatre bandes** plutôt que les trois couches classiques (présentation, métier, données). Dans un système réparti, la frontière réseau est une couche à part entière : elle fixe ce que le serveur expose et sous quelle forme. Les quatre bandes sont les suivantes :

Bande	Contenu	Pourquoi elle est séparée
Client / présentation	Ce que voit l'utilisateur : console, Postman, scripts de test	c'est l'autre extrémité du client-serveur
Point d'entrée serveur / distribution	Ce qui traduit le protocole en appels : boucle `accept()` et analyse du texte, interfaces `Remote` et registre, `@RestController` et DTO	c'est la « présentation » du serveur : elle seule devrait connaître le protocole
Métier	Calcul, règles (division par zéro, types d'opération)	c'est ce qui doit rester identique d'un module à l'autre
Données / état	historique, stockage en mémoire	même en mémoire, c'est un état partagé, avec sa concurrence et son cycle de vie

Les trois modules réalisent le **même besoin** (calculatrice et historique), ce qui permet de les comparer bande par bande. Un composant est placé dans la bande qui correspond à ce que **fait son code**, et non à ce que suggère son nom. Quand une classe couvre plusieurs bandes, le schéma la dessine **à cheval** sur ces bandes plutôt que de la découper artificiellement (`CalcServer`, `OperationService`).

2. B.1 Inventaire des couches réellement présentes

`/socket` aucune séparation de couches

Bande	Ce qui existe vraiment
Client	`CalcClient` : lit une ligne au clavier et l'envoie sans validation.
Point d'entrée	`CalcServer.main()` / `CalcServer3.ClientHandler.run()` : `accept()`, flux, un thread par connexion (version 3).
Métier	`calculate()`, **la même méthode** qui analyse aussi le protocole (`split(";")`, `parseDouble`) et fabrique la réponse `"RESULT;" + x`.
Données	Rien : aucun état n'est conservé. Le journal se limite à `System.out`.

Constat honnête : le module est **une seule couche**. Ce n'était pas l'objectif de la séance 1, centrée sur les sockets. La logique de calcul est en outre copiée à l'identique dans deux classes.

`/rmi` des couches identifiables, mais toutes liées à RMI

Bande	Ce qui existe vraiment
Client	`Client` (menu console), `ConcurrentLoadTest`, `BenchmarkRMI`, `ClientNotificationListener` (objet distant de rappel).
Point d'entrée	Registre `:1099`, interfaces `CalculatorService` et `HistoryService` (`extends Remote`).
Métier	`CalculatorServiceImpl` : calcul et enregistrement de l'historique. La classe **est** un objet RMI (`extends UnicastRemoteObject`).
Données	`HistoryServiceImpl` : `ArrayList` protégée par `synchronized`, **plus** la diffusion des notifications aux clients.

La séparation métier / données existe : deux classes, et la dépendance `Calculator → History` est injectée dans `Server.java`. En revanche, chaque couche dépend de la technologie de distribution, et le client accède directement à la couche basse.

`/rest` le module le plus propre, à qui il manque une couche

Bande	Ce qui existe vraiment
Client	Postman (collection), `benchmark-rest.ps1`.
Point d'entrée	`OperationController` (`@RestController`) : validation de présence, conversion des exceptions en 400 / 422 / 404, `201 Created` et `Location` ; DTO `OperationRequest` et `OperationDTO`.
Métier	`OperationService` (`@Service`) : normalisation, calcul, refus de `b == 0`.
Données	**Pas de couche dédiée** : `List<Operation>`, `AtomicLong` et `synchronized` vivent dans `OperationService`. Le modèle `Operation` existe.

Le sens des dépendances est respecté : `OperationService` n'importe aucun type HTTP (`ResponseEntity`, `HttpStatus`…). Les erreurs métier sont signalées par des exceptions Java standard, que seul le contrôleur traduit en codes HTTP.

3. B.2 Violations trouvées

Les repères (S, R, E) renvoient au schéma, où chaque violation est annotée.

Id	Catégorie (B.2)	Où	Risque concret	Cause probable
S1	Absence de couches	`CalcServer.java:49`, `calcServer3.java:108`	Le calcul fabrique la trame `RESULT;` : changer de protocole oblige à réécrire le calcul.	Séance 1 centrée sur les sockets (choix assumé)
S2	Logique dupliquée	`calculate()` en deux copies	Une correction faite dans une seule copie fait diverger les deux serveurs.	Copie pour créer la version multi-thread (délais)
R1	**Saut de couche**	`Client.java:22,29,126` ; `ConcurrentLoadTest.java:23,117` ; `record()` publié dans `HistoryService.java:11`	Le client contourne `CalculatorService`. N'importe quel client du registre peut appeler `record("ADD(1,1) = 3")` et **falsifier l'historique**, y compris le comptage sur lequel repose `ConcurrentLoadTest`.	Callbacks ajoutés au plus court (séance 3)

Id	Catégorie (B.2)	Où	Risque concret	Cause probable
R2	**Couche basse qui connaît la couche haute**	`HistoryService.java:3`, `HistoryServiceImpl.java:3` → `import client.NotificationListener`	Le paquet `server` ne compile pas sans le paquet `client` : impossible de livrer un serveur seul.	Interface rangée à côté de son implémentation (oubli)
R3	**Couche basse qui connaît une technologie haute**	`CalculatorServiceImpl extends UnicastRemoteObject`, `throws RemoteException` partout	Le calcul ne peut être ni réutilisé par REST ou SOAP, ni testé sans registre RMI.	Forme du tutoriel RMI, jamais remise en question
R4	Détour par la distribution	`Server.java:22,29` injecte le **stub** obtenu par `lookup`	Chaque `add()` refait un aller-retour RMI en boucle locale. La `RemoteException` est avalée (`CalculatorServiceImpl.java:27`), et l'entrée est perdue sans erreur visible.	Copier-coller du code de `lookup` côté client
R5	Couplage à l'exécution	`HistoryServiceImpl.java:40`	`record()` appelle chaque client abonné **avant** de rendre la main : la durée d'un `add()` dépend du client le plus lent.	Simplicité du callback synchrone (choix à courte vue)
E1	Couche manquante (fusion)	`OperationService.java: 16-19, 79, 88, 103`	Passer à une base de données oblige à réécrire le service, pas seulement à ajouter un repository.	Pas de persistance exigée (choix à courte vue)
E2	Responsabilité dupliquée	`OperationController.java:31-34` et `OperationService.java: 26`	La même règle (type non vide) produit deux messages différents selon l'endroit où elle échoue, et ces deux versions risquent de diverger.	Propriétaire de la règle jamais choisi
E3	Symptôme « lasagne »	`Operation` ≡ `OperationDTO`, `toDTO()` l. 118	Ajouter un champ oblige à toucher quatre endroits sans aucune transformation.	Patron DTO appliqué par réflexe

Méthodes de pure délégation : 0 au sens strict. Trois candidates ont été examinées :

- `OperationController.getAllOperations()` se contente d'envelopper le résultat en `200 OK`. C'est la traduction HTTP attendue d'un contrôleur, donc une indirection justifiée.
- `CalculatorServiceImpl.recordHistory()` ajoute un contrôle de `null` et une gestion d'erreur.
- `HistoryServiceImpl.getAll()` renvoie une copie défensive.

Le projet souffre donc plutôt du défaut **inverse** de la lasagne, c'est-à-dire de couches manquantes (S1, E1). Le seul symptôme lasagne observé est la duplication de structure entre `Operation` et `OperationDTO` (E3).

4. Décision : corriger ou assumer ? (violation choisie : R1)

Rappel. Le client appelle `HistoryService` directement, ce qui est une couche détendue (*relaxed layering*). L'interface distante expose aussi `record()`, c'est-à-dire l'écriture.

Cette violation mélange deux problèmes dont le rapport coût / bénéfice est très différent. Elle est donc traitée en deux parties.

a) Écriture exposée (`record()` publiée dans le registre) à corriger maintenant

- **Bénéfice.** C'est le seul cas où la violation crée un **risque réel** : n'importe quel client peut écrire de fausses entrées dans l'historique. L'intégrité de l'historique est la propriété que le test `ConcurrentLoadTest` prétend vérifier. Un historique falsifiable rend ce test non probant.
- **Coût.** Environ quinze lignes dans quatre fichiers, sans aucun changement côté client :

  - retirer `record()` de l'interface distante `HistoryService` ;
  - créer une interface locale `HistoryRecorder { void record(String) }`, implémentée par `HistoryServiceImpl` ;
  - dans `Server.java`, injecter `historyImpl` (l'objet local) au lieu du stub ;
  - changer le type du champ dans `CalculatorServiceImpl`.
- **Bonus.** La même modification supprime **R4** : plus d'aller-retour RMI local, plus de `RemoteException` avalée.

Le coût est faible et le risque est concret : la correction est justifiée tout de suite.

b) Lecture directe (`getAll()`, `subscribe()`) assumée consciemment pour l'instant

- **Coût de la correction.** Faire de `CalculatorService` la seule porte d'entrée oblige à ajouter `getHistory()` et `subscribe()` à `CalculatorService`, et à modifier `Client`, `ConcurrentLoadTest` et `Server`. Les deux nouvelles méthodes ne feraient que **déléguer** à `HistoryService` sans rien décider : c'est exactement le symptôme lasagne vu en cours.
- **Bénéfice.** Il est nul aujourd'hui. Lire l'historique ne fait intervenir **aucune règle métier** : ni droits, ni filtrage, ni calcul. Pour cette couche candidate, la question du cours (« que ferait-on de moins bien si elle n'existait pas ? ») a pour réponse honnête « rien ».
- **Condition de remise en cause.** Dès qu'une règle apparaît sur la lecture (historique par utilisateur, droits d'accès, pagination, persistance en base), la façade gagne une vraie responsabilité. On la crée alors, et on supprime l'accès direct.

En résumé. Corriger l'écriture tout de suite, parce qu'elle coûte peu et supprime un risque. Assumer la lecture directe comme dette technique **documentée**, avec sa condition de remboursement, parce que la supprimer ajouterait aujourd'hui de la complexité sans valeur.